

EXTRACCIÓN, TRANSFORMACIÓN Y LIMPIEZA DE DATOS
ETL CON CURSORES EN SQL SERVER Y PAQUETES SSIS

HENRY ALEJANDRO ORTEGA RODRIGUEZ

JOSÉ RODOLFO HERNÁNDEZ SUÁREZ

UNIVERSIDAD DE CUNDINAMARCA
INGENIERÍA DE SISTEMAS Y COMPUTO

BIG DATA

CUNDINAMARCA 2026

Enlaces de la entrega

Repositorio (scripts SQL y proyecto SSIS): <https://github.com/AdrenMii/bigdata-actividad4-etl-ssis>

Video de demostración: [PEGAR AQUÍ EL ENLACE DEL VIDEO]

Integrantes: Henry Alejandro Ortega Rodriguez y José Rodolfo Hernández Suárez.

Introducción

Este informe documenta la construcción de un proceso de extracción, transformación y carga (ETL) a partir del archivo `PersonasETLActividad4.xlsx`, que contiene 1.000 registros ficticios de personas encuestadas con errores introducidos a propósito: documentos vacíos, importes escritos en letras o con formatos distintos, fechas imposibles, categorías mal escritas, incoherencias entre municipio y departamento y registros duplicados.

El trabajo tiene tres partes. Primero, un ejercicio con cursores en SQL Server que limpia los registros 1 a 100 y calcula el balance mensual. Segundo, un paquete de SQL Server Integration Services (SSIS) que lleva los 1.000 registros del Excel a un modelo relacional, aplicando reglas de calidad, separando rechazos y resolviendo duplicados. Tercero, un segundo paquete que toma solo la información depurada del modelo relacional y la carga en un data warehouse con esquema estrella, sobre el cual se ejecutan cuatro consultas de análisis.

Objetivo

Implementar procesos de extracción, transformación y carga mediante cursores en SQL Server y componentes de SSIS, aplicando tratamientos de calidad para trasladar información desde Excel hacia un modelo relacional y, posteriormente, hacia un modelo multidimensional.

Herramientas utilizadas

- SQL Server 2022 Developer y SQL Server Management Studio 22.

- Visual Studio 2022 Community con la extensión SQL Server Integration Services Projects.
- Microsoft Access Database Engine (ACE OLE DB 16.0) para leer el Excel.
- draw.io para los diagramas y GitHub para el repositorio.

Se crearon dos bases de datos: PersonasMER (modelo relacional, destino del primer paquete) y PersonasDW (data warehouse, destino del segundo paquete). El Excel original se conserva intacto en la carpeta datos-original del repositorio y no se hizo ninguna limpieza manual.

Ejercicio de ETL mediante cursores en SQL Server

El ejercicio toma los registros cuyo SourceRowId está entre 1 y 100. El script `sql/03_EjercicioCursor.sql` sigue estos pasos:

1. **Staging.** Con OPENROWSET y el proveedor ACE se leen las filas 1 a 100 del Excel y se guardan tal cual en la tabla CursorStaging, conservando los valores originales.
2. **Recorrido con cursor.** Se declara el cursor `cur_personas` sobre la tabla de staging, se abre con OPEN, se lee cada fila con FETCH NEXT y se repite con WHILE `@@FETCH_STATUS = 0`. Al final se cierra con CLOSE y se libera de memoria con DEALLOCATE.
3. **Limpieza de textos.** En cada fila se quitan los espacios sobrantes (LTRIM, RTRIM y reemplazo de dobles espacios), el tipo de documento pasa a mayúsculas y los nombres y apellidos quedan con la primera letra en mayúscula.
4. **Campos obligatorios.** Se identifican los registros sin número de documento (vacío o SIN-DATO), sin primer nombre o sin primer apellido.

5. **Importes.** En MonthlyIncome y MonthlyExpenses se quitan el símbolo \$ y los espacios; como el enunciado indica que en los formatos alternativos el punto es separador de miles y la coma es decimal, se eliminan los puntos y la coma se cambia por punto. Luego se convierte con TRY_CONVERT a DECIMAL(18,2). Si el valor falta, no es numérico o es negativo, se registra el motivo.
6. **Balance.** Para los registros válidos se calcula ingreso – gasto. Un balance negativo es válido y no causa rechazo; un importe desconocido nunca se reemplaza por cero.
7. **Destino.** Los registros válidos se guardan en CursorAceptados (con el valor original y el convertido) y los rechazados en CursorRechazados, con el identificador de origen y el motivo.

Tabla 1*Resultado del ejercicio con cursor*

Procesados	Aceptados	Rechazados	Aceptados con balance negativo
100	78	22	30

Nota. Conteos obtenidos al ejecutar el script; los mismos que muestra el PRINT final del cursor.

Tabla 2*Registros rechazados por el cursor y su motivo*

Fila	Motivo	Fila	Motivo
2	Sin numero de documento	55	Ingreso negativo (-500000)
3	Sin numero de documento	56	Ingreso no numerico (dos millones)
13	Ingreso no numerico (dos millones)	57	Gasto faltante
19	Ingreso negativo (-500000)	67	Sin numero de documento
20	Ingreso no numerico (dos millones)	69	Sin primer nombre
21	Gasto faltante	74	Sin numero de documento
31	Sin numero de documento	75	Sin numero de documento
33	Sin primer nombre	85	Ingreso no numerico (dos millones)
38	Sin numero de documento	91	Ingreso negativo (-500000)
39	Sin numero de documento	92	Ingreso no numerico (dos millones)
49	Ingreso no numerico (dos millones)	93	Gasto faltante

Tabla 3*Correcciones de importes hechas por el cursor*

Fila	Ingreso original	Ingreso	Gasto original	Gasto	Balance
18	\$ 14.700.000,00	14.700.000,00	2600000	2.600.000,00	12.100.000,00
22	5300000	5.300.000,00	8900000,50	8.900.000,50	-3.600.000,50
54	\$ 10.500.000,00	10.500.000,00	11200000	11.200.000,00	-700.000,00
58	1800000	1.800.000,00	500000,50	500.000,50	1.299.999,50
90	\$ 5.200.000,00	5.200.000,00	3600000	3.600.000,00	1.600.000,00
94	1100000	1.100.000,00	4600000,50	4.600.000,50	-3.500.000,50

Nota. El balance negativo de las filas 22, 54 y 94 es válido: la persona gasta más de lo que gana.

Tabla 4*Correcciones de texto hechas por el cursor*

Fila	Tipo doc. antes	Después	Nombre antes	Después	Apellido antes	Después
1	PA	PA	andrés	Andrés	GARCÍA	García
4	cc	CC	Laura	Laura	López	López
37	CC	CC	camila	Camila	MARTÍNEZ	Martínez
40	cc	CC	Andrés	Andrés	Díaz	Díaz
73	CC	CC	valentina	Valentina	CAÑÓN	Cañón
76	pa	PA	Ana	Ana	Díaz	Díaz

Código del cursor

```

/* =====
Actividad 4 - ETL | 03: Ejercicio de ETL mediante cursores
Registros con SourceRowId entre 1 y 100:
1. Se importan a una tabla de staging sin cambiar nada.
2. Un cursor los recorre uno por uno: limpia textos, convierte los
   importes, valida y calcula el balance mensual.
3. Los validos van a CursorAceptados y los rechazados a
   CursorRechazados con el motivo.
Reglas: un importe desconocido NO se reemplaza por cero, y un balance
negativo es valido (no causa rechazo).
===== */

USE PersonasMER;
GO

/* ----- Requisito: permitir leer el Excel desde SQL Server ----- */
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'Ad Hoc Distributed Queries', 1; RECONFIGURE;
EXEC master.dbo.sp_MSset_oledb_prop N'Microsoft.ACE.OLEDB.16.0', N'AllowInProcess', 1;
EXEC master.dbo.sp_MSset_oledb_prop N'Microsoft.ACE.OLEDB.16.0', N'DynamicParameters', 1;
GO

/* ----- 1. Staging: copia fiel de las filas 1 a 100 ----- */
DROP TABLE IF EXISTS dbo.CursorStaging;

SELECT *
INTO dbo.CursorStaging
FROM OPENROWSET('Microsoft.ACE.OLEDB.16.0',
'Excel 12.0 Xml;HDR=YES;IMEX=1;Database=C:\Users\henry\Desktop\Universidad\Big Data\Actividad 4 - ETL
SSIS\datos-original\PersonasETLActividad4.xlsx',
'SELECT * FROM [Personas$]')
WHERE TRY_CAST(SourceRowId AS INT) BETWEEN 1 AND 100;

```

```

/* ----- Tablas de destino ----- */
DROP TABLE IF EXISTS dbo.CursorAceptados;
DROP TABLE IF EXISTS dbo.CursorRechazados;

CREATE TABLE dbo.CursorAceptados (
    SourceRowId INT NOT NULL PRIMARY KEY,
    TipoDocumento VARCHAR(5) NOT NULL,
    NumeroDocumento VARCHAR(20) NOT NULL,
    PrimerNombre NVARCHAR(50) NOT NULL,
    PrimerApellido NVARCHAR(50) NOT NULL,
    IngresoOriginal NVARCHAR(255) NULL, -- como venia en el Excel
    GastoOriginal NVARCHAR(255) NULL,
    IngresoMensual DECIMAL(18,2) NOT NULL, -- ya convertido
    GastoMensual DECIMAL(18,2) NOT NULL,
    BalanceMensual DECIMAL(18,2) NOT NULL
);

CREATE TABLE dbo.CursorRechazados (
    SourceRowId INT NOT NULL PRIMARY KEY,
    Motivo NVARCHAR(400) NOT NULL
);
GO

/* ----- 2. Cursor ----- */
SET NOCOUNT ON;

DECLARE @SourceRowId INT,
        @TipoDoc NVARCHAR(255), @NumDoc NVARCHAR(255),
        @Nombre NVARCHAR(255), @Apellido NVARCHAR(255),
        @IngresoTxt NVARCHAR(255), @GastoTxt NVARCHAR(255);

DECLARE @Ingreso DECIMAL(18,2), @Gasto DECIMAL(18,2),
        @Limpio NVARCHAR(255), @Motivo NVARCHAR(400),
        @Procesados INT = 0, @Aceptados INT = 0, @Rechazados INT = 0;

-- DECLARACION del cursor
DECLARE cur_personas CURSOR LOCAL FAST_FORWARD FOR
    SELECT CAST(SourceRowId AS INT), DocumentType, DocumentNumber,
           FirstName, LastName, MonthlyIncome, MonthlyExpenses
    FROM dbo.CursorStaging
    ORDER BY CAST(SourceRowId AS INT);

-- APERTURA
OPEN cur_personas;

-- LECTURA del primer registro
FETCH NEXT FROM cur_personas
    INTO @SourceRowId, @TipoDoc, @NumDoc, @Nombre, @Apellido, @IngresoTxt, @GastoTxt;

-- RECORRIDO: mientras la lectura sea exitosa
WHILE @@FETCH_STATUS = 0
BEGIN
    SET @Procesados += 1;
    SET @Motivo = N'';

    -- a) Limpieza de textos: quitar espacios sobrantes y estandarizar mayusculas
    SET @TipoDoc = UPPER(LTRIM(RTRIM(@TipoDoc)));
    SET @NumDoc = UPPER(LTRIM(RTRIM(@NumDoc)));
    SET @Nombre = LTRIM(RTRIM(@Nombre));
    SET @Apellido = LTRIM(RTRIM(@Apellido));
    WHILE CHARINDEX(N' ', @Nombre) > 0 SET @Nombre = REPLACE(@Nombre, N' ', N' ');
    WHILE CHARINDEX(N' ', @Apellido) > 0 SET @Apellido = REPLACE(@Apellido, N' ', N' ');
    -- Primera letra mayuscula y el resto minuscula ("andr s" / "GARC A" -> "Andr s" / "Garc a")
    SET @Nombre = UPPER(LEFT(@Nombre, 1)) + LOWER(SUBSTRING(@Nombre, 2, 255));
    SET @Apellido = UPPER(LEFT(@Apellido, 1)) + LOWER(SUBSTRING(@Apellido, 2, 255));

    -- b) Campos obligatorios
    IF @NumDoc IS NULL OR @NumDoc = N'' OR @NumDoc = N'SIN-DATO'
        SET @Motivo += N'Sin numero de documento; ';
    IF @Nombre IS NULL OR @Nombre = N''

```

```

        SET @Motivo += N'Sin primer nombre; ';
    IF @Apellido IS NULL OR @Apellido = N''
        SET @Motivo += N'Sin primer apellido; ';

    -- c) Ingreso: quitar $ y espacios; el punto es separador de miles y la coma es decimal
    SET @Ingreso = NULL;
    IF @IngresoTxt IS NULL OR LTRIM(RTRIM(@IngresoTxt)) = N''
        SET @Motivo += N'Ingreso faltante; ';
    ELSE
    BEGIN
        SET @Limpio = REPLACE(REPLACE(REPLACE(REPLACE(@IngresoTxt, N'$ ', N''), N' ', N''), N'.'. , N''),
N',', N'.'');
        SET @Ingreso = TRY_CONVERT(DECIMAL(18,2), @Limpio);
        IF @Ingreso IS NULL
            SET @Motivo += N'Ingreso no numerico (' + @IngresoTxt + N'); ';
        ELSE IF @Ingreso < 0
            SET @Motivo += N'Ingreso negativo (' + @IngresoTxt + N'); ';
    END

    -- d) Gasto: mismo tratamiento
    SET @Gasto = NULL;
    IF @GastoTxt IS NULL OR LTRIM(RTRIM(@GastoTxt)) = N''
        SET @Motivo += N'Gasto faltante; ';
    ELSE
    BEGIN
        SET @Limpio = REPLACE(REPLACE(REPLACE(REPLACE(@GastoTxt, N'$ ', N''), N' ', N''), N'.'. , N''), N',',
N'.'');
        SET @Gasto = TRY_CONVERT(DECIMAL(18,2), @Limpio);
        IF @Gasto IS NULL
            SET @Motivo += N'Gasto no numerico (' + @GastoTxt + N'); ';
        ELSE IF @Gasto < 0
            SET @Motivo += N'Gasto negativo (' + @GastoTxt + N'); ';
    END

    -- e) Destino: aceptado (con balance) o rechazado (con motivo)
    IF @Motivo = N''
    BEGIN
        INSERT INTO dbo.CursorAceptados
            (SourceRowId, TipoDocumento, NumeroDocumento, PrimerNombre, PrimerApellido,
            IngresoOriginal, GastoOriginal, IngresoMensual, GastoMensual, BalanceMensual)
        VALUES
            (@SourceRowId, @TipoDoc, @NumDoc, @Nombre, @Apellido,
            @IngresoTxt, @GastoTxt, @Ingreso, @Gasto, @Ingreso - @Gasto); -- balance = ingreso - gasto
        SET @Aceptados += 1;
    END
    ELSE
    BEGIN
        INSERT INTO dbo.CursorRechazados (SourceRowId, Motivo)
        VALUES (@SourceRowId, LEFT(@Motivo, LEN(@Motivo) - 1));
        SET @Rechazados += 1;
    END

    -- Siguiente registro
    FETCH NEXT FROM cur_personas
    INTO @SourceRowId, @TipoDoc, @NumDoc, @Nombre, @Apellido, @IngresoTxt, @GastoTxt;
END

-- CIERRE y LIBERACION de memoria
CLOSE cur_personas;
DEALLOCATE cur_personas;

/* ----- 3. Resultados ----- */
PRINT CONCAT('Procesados: ', @Procesados, ' | Aceptados: ', @Aceptados, ' | Rechazados: ', @Rechazados);

SELECT @Procesados AS Procesados, @Aceptados AS Aceptados, @Rechazados AS Rechazados;
GO

-- Rechazados con su motivo
SELECT SourceRowId, Motivo FROM dbo.CursorRechazados ORDER BY SourceRowId;

-- Ejemplos de correccion: importes que venian con otro formato

```

```

SELECT SourceRowId, IngresoOriginal, IngresoMensual, GastoOriginal, GastoMensual, BalanceMensual
FROM dbo.CursorAceptados
WHERE IngresoOriginal LIKE N'%$%' OR GastoOriginal LIKE N'%,%'
ORDER BY SourceRowId;

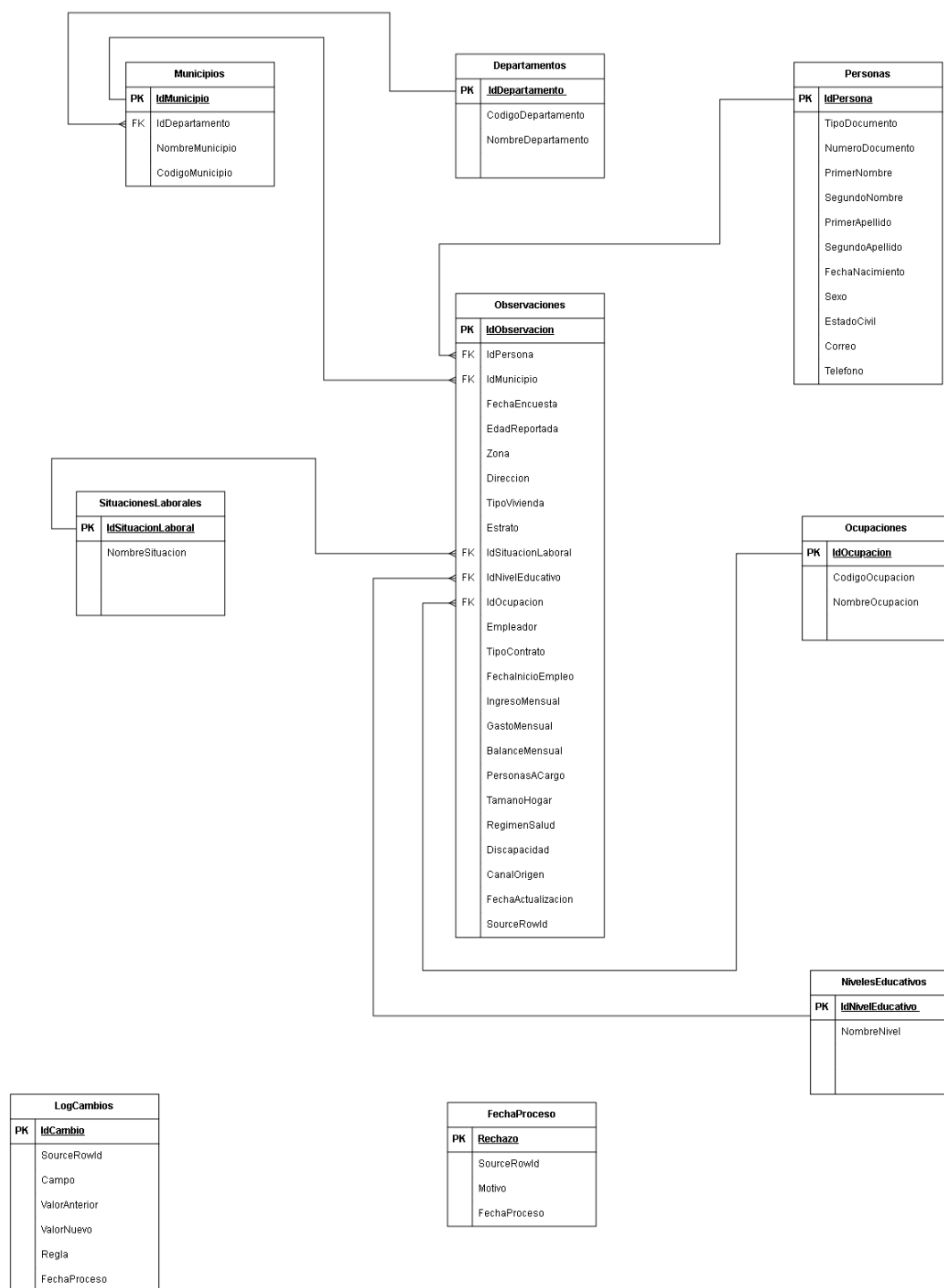
-- Balances negativos (validos, no se rechazan)
SELECT COUNT(*) AS AceptadosConBalanceNegativo FROM dbo.CursorAceptados WHERE BalanceMensual < 0;

```

Modelo relacional

El modelo relacional separa la **persona** de la **observación** (la encuesta), porque una misma persona puede responder varias veces. La persona se identifica por tipo y número de documento (restricción UNIQUE), y la observación por la persona junto con la fecha de encuesta. Los municipios, departamentos, niveles educativos, situaciones laborales y ocupaciones quedan en tablas de catálogo para no repetir texto ni arrastrar errores de escritura. Las tablas Rechazos y LogCambios guardan, respectivamente, los registros que no se cargaron con su motivo y los cambios aplicados a los datos.

Figura 1
Diagrama del modelo relacional (PersonasMER)

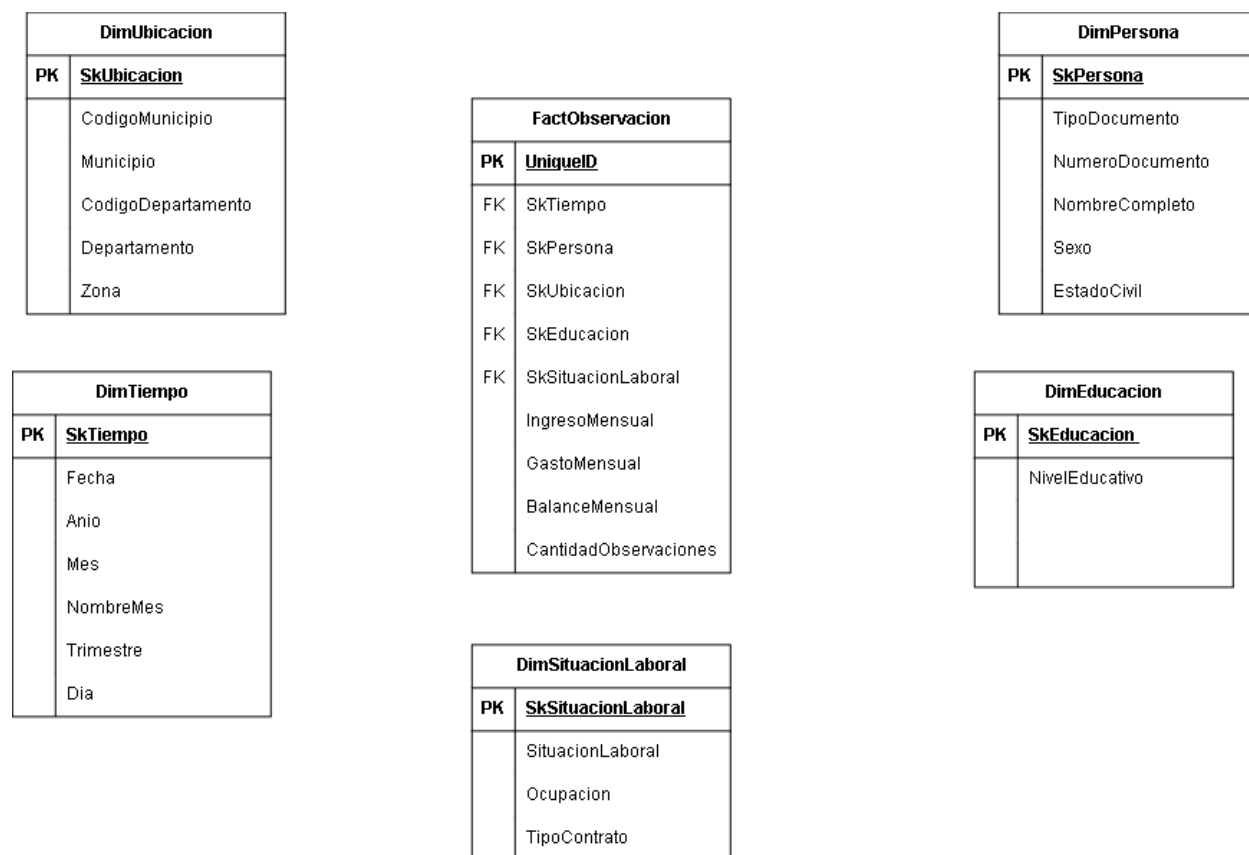


Además de las tablas del diagrama, la base tiene una tabla de staging (StgPersonas) donde el primer flujo deja los datos ya limpios y sin duplicados, y una tabla ResumenCargas con los conteos de cada ejecución. Los catálogos de departamentos y municipios se llenan con los

códigos internos del propio dataset (D01 a D05 y M001 a M010) y funcionan como **datos maestros de referencia**: con ellos se valida la coherencia entre municipio y departamento y se estandariza el nombre del municipio. El código M999 MUNICIPIO DESCONOCIDO no se incluye a propósito, para que esas filas queden en revisión en lugar de inventarles una ubicación.

Modelo multidimensional

El data warehouse PersonasDW usa un esquema estrella. La tabla de hechos FactObservacion tiene como **grano una observación por persona y fecha de encuesta**, garantizado con una restricción UNIQUE (SkPersona, SkTiempo). Sus medidas son ingreso mensual, gasto mensual, balance mensual y cantidad de observaciones. Las dimensiones permiten analizar por tiempo (DimTiempo: fecha, año, mes, nombre del mes, trimestre y día), persona (DimPersona), ubicación (DimUbicacion: municipio, departamento y zona), educación (DimEducacion) y situación laboral (DimSituacionLaboral: situación, ocupación y tipo de contrato). Cada dimensión tiene una clave sustituta (Sk) propia del data warehouse, y la tabla de hechos se relaciona con ellas mediante búsquedas (Lookup) en SSIS.

Figura 2*Diagrama del modelo estrella (PersonasDW)***Correspondencia entre el Excel y el modelo relacional****Tabla 5***Columnas del Excel y su destino en PersonasMER*

Columna del Excel	Destino	Tratamiento
SourceRowId	Observaciones.SourceRowId	Entero; identifica la fila de origen (también en Rechazos y LogCambios)
DocumentType	Personas.TipoDocumento	Sin espacios y en mayúsculas
DocumentNumber	Personas.NumeroDocumento	Mayúsculas; vacío o SIN-DATO = rechazo
FirstName / MiddleName	Personas.PrimerNombre / SegundoNombre	Primera letra en mayúscula; primer nombre obligatorio
LastName / SecondLastName	Personas.PrimerApellido / SegundoApellido	Primera letra en mayúscula; primer apellido obligatorio
BirthDate	Personas.FechaNacimiento	dd/mm/aaaa → aaaa-mm-dd; inválida = revisión
ReportedAge	Observaciones.EdadReportada	Se compara con la edad calculada
Sex	Personas.Sexo	Solo M, F o ND; otro valor = NULL
MaritalStatus	Personas.EstadoCivil	Mayúsculas
Email	Personas.Correo	Sin @ = NULL
Phone	Personas.Telefono	Sin guiones; distinto de 10 dígitos = NULL

Columna del Excel	Destino	Tratamiento
DepartmentCode / DepartmentName	Departamentos (vía Municipios)	Se toma del catálogo según el municipio
MunicipalityCode / MunicipalityName	Observaciones.IdMunicipio → Municipios	Lookup por código; M999 = revisión
Zone	Observaciones.Zona	Solo URBANA o RURAL; otro valor = NULL
Address	Observaciones.Direccion	Sin espacios sobrantes
HousingType	Observaciones.TipoVivienda	Mayúsculas
SocioeconomicStratum	Observaciones.Estrato	Solo 1 a 6; otro valor = NULL
EducationLevel	Observaciones.IdNivelEducativo → NivelesEducativos	Mayúsculas y Lookup
EmploymentStatus	Observaciones.IdSituacionLaboral → SituacionesLaborales	Mayúsculas y Lookup
OccupationCode / OccupationName	Observaciones.IdOcupacion → Ocupaciones	Lookup; sin ocupación = NULL
EmployerName	Observaciones.Empleador	Sin espacios sobrantes
ContractType	Observaciones.TipoContrato	Mayúsculas; EMPLEADO con NO_APLICA = revisión
EmploymentStartDate	Observaciones.FechaInicioEmpleo	Posterior a la encuesta = NULL
MonthlyIncome	Observaciones.IngresoMensual	Sin \$ ni puntos; coma decimal; faltante, texto o negativo = rechazo
MonthlyExpenses	Observaciones.GastoMensual	Igual que el ingreso
(calculado)	Observaciones.BalanceMensual	Ingreso – gasto (puede ser negativo)
Dependents	Observaciones.PersonasACargo	Entero ≥ 0 ; 2.5, "muchos" o -2 = NULL
HouseholdSize	Observaciones.TamanoHogar	Entero ≥ 1 ; 0 = NULL
HealthRegime	Observaciones.RegimenSalud	NULL, "sin dato" y N/A = NULL
Disability	Observaciones.Discapacidad	sí/1 = SI, no/0 = NO, ND
SurveyDate	Observaciones.FechaEncuesta	Fecha inválida = rechazo (es parte de la clave)
UpdatedAt	Observaciones.FechaActualizacion	Decide qué versión se conserva en un duplicado
SourceChannel	Observaciones.CanalOrigen	Mayúsculas

Diagnóstico inicial y reglas de tratamiento

Antes de construir los paquetes se revisaron las 37 columnas del Excel para encontrar valores faltantes, duplicados, formatos inconsistentes y datos inválidos. Todas las columnas llegan como texto. Los hallazgos principales fueron los siguientes:

Tabla 6

Diagnóstico inicial del Excel (1.000 registros)

Problema encontrado	Registros
Tipo de documento en minúsculas (cc, pa, ce)	23
Número de documento vacío o con el texto SIN-DATO	68

Problema encontrado	Registros
Primer nombre vacío / nombres en minúsculas / apellidos en mayúsculas	22 / 23 / 23
Fecha de nacimiento con formato dd/mm/aaaa	68
Fecha de nacimiento imposible (31/02/1990) o posterior a la encuesta (2035)	45 / 23
Edad reportada que no coincide con la fecha de nacimiento (desfase de 12 años)	46
Sexo con valor Z (fuera de M, F, ND)	22
Estado civil y nivel educativo en minúsculas	44 / 44
Correo sin @ / teléfono con guion / teléfono de 3 dígitos	22 / 45 / 22
Municipio en minúsculas o sin tilde	22
Departamento que no corresponde al municipio	31
Municipio desconocido (M999)	45
Zona CENTRO / estrato 0, 9 o ALTO	22 / 22
Ingreso escrito en letras ("dos millones") / negativo / con formato \$ 14.700.000,00	44 / 22 / 22
Gasto vacío / con coma decimal (8900000,50)	22 / 22
Personas a cargo 2.5, "muchos" o -2 / tamaño del hogar 0	44 / 22
Régimen de salud NULL, "sin dato" o N/A / discapacidad 0, 1, no, sí	22 / 44
Fecha de encuesta imposible (2026-13-40)	22
EMPLEADO sin contrato (NO_APLICA)	71
Inicio de empleo posterior a la encuesta (2028)	22
Filas 951 a 1000 que repiten las filas 801 a 850	50

Nota. Los documentos con formato SIM más siete dígitos, los teléfonos que empiezan por 000 y los códigos geográficos internos no se consideraron errores, tal como lo indica el enunciado. Muchos errores caen en las mismas filas, por eso la suma de la tabla es mayor que el número de registros afectados.

Con base en el diagnóstico se definieron tres tipos de tratamiento:

- **Rechazo:** el registro no puede cargarse porque le falta algo indispensable:
documento, primer nombre o primer apellido (clave de la persona), ingreso o gasto válido (medidas del análisis) o fecha de encuesta válida (clave de la observación). No se inventa ningún valor para completarlo.
- **Revisión:** el registro tiene una inconsistencia que una persona debe revisar antes de usarlo: municipio desconocido, fecha de nacimiento inválida o posterior a la encuesta, edad reportada que difiere en más de un año de la calculada, o persona EMPLEADA sin tipo de contrato. Tampoco se carga y queda en la tabla Rechazos con el prefijo REVISIÓN.

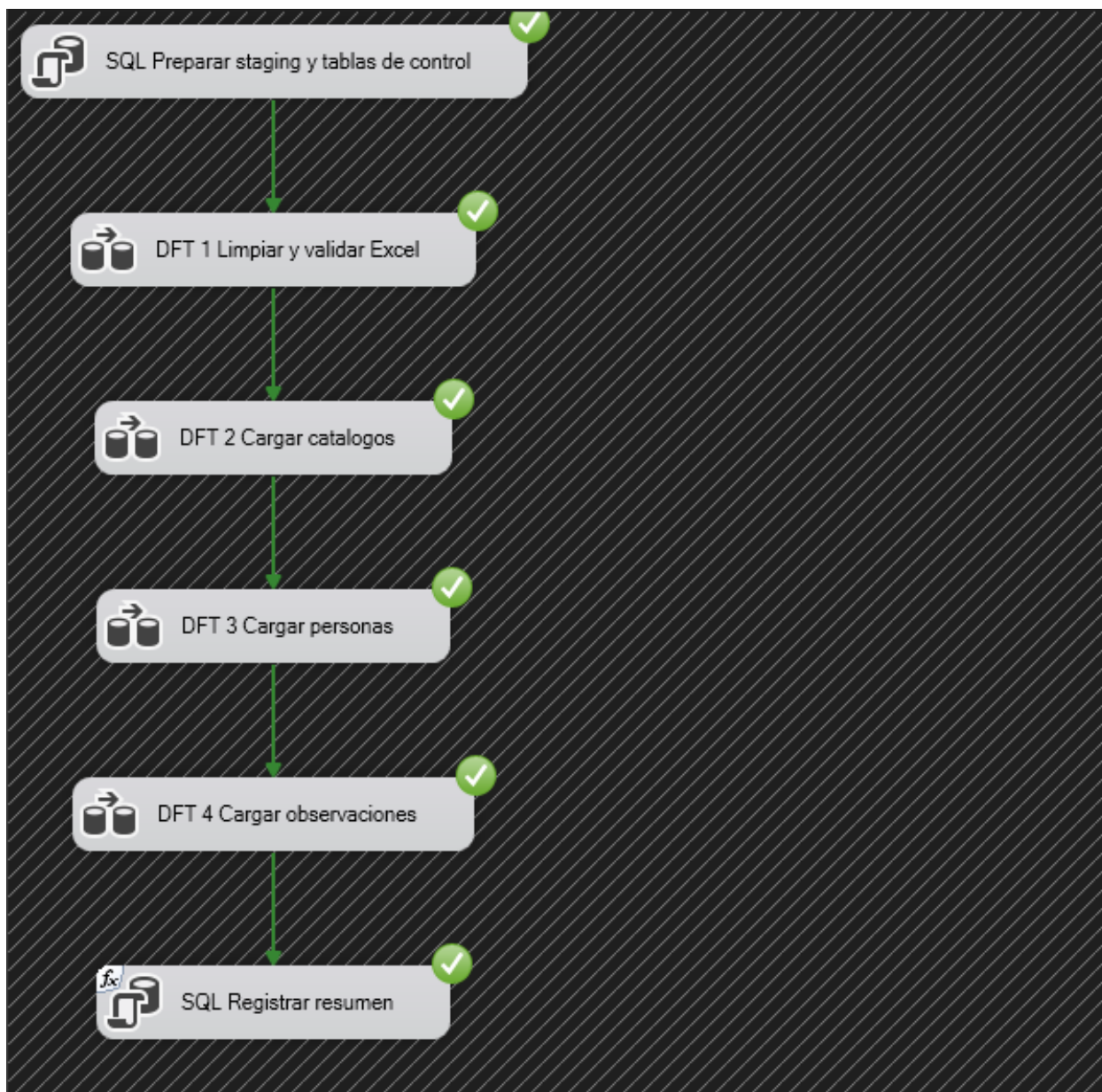
- **Corrección o valor nulo:** el registro se carga. Los formatos se corrigen (mayúsculas, importes, fechas, municipio, departamento) y los datos opcionales inválidos (correo, teléfono, zona, estrato, sexo Z, personas a cargo, tamaño del hogar, régimen de salud, inicio de empleo futuro) quedan en NULL en lugar de inventarse.

Duplicados. Se usa el tipo y número de documento como clave de persona, y la persona junto con la fecha de encuesta como clave de observación. Cuando hay dos versiones de la misma observación se conserva la que tiene la **fecha de actualización ('UpdatedAt')** más reciente, porque representa el último dato reportado; si las dos versiones son idénticas, se conserva una sola.

Paquete SSIS A: Excel al modelo relacional

El paquete `PaqueteA_ExcelARelacional.dtsx` tiene seis tareas en el flujo de control, que se ejecutan en orden:

1. **SQL Preparar staging y tablas de control:** vacía la staging, Rechazos y LogCambios, que guardan solo lo de la última ejecución. No borra los datos cargados.
2. **DFT 1 Limpiar y validar Excel:** lee el Excel, limpia, valida, separa rechazos y revisión, registra los cambios y resuelve duplicados.
3. **DFT 2 Cargar catálogos:** carga niveles educativos, situaciones laborales y ocupaciones nuevos.
4. **DFT 3 Cargar personas:** una persona por documento, solo las que no existen.
5. **DFT 4 Cargar observaciones:** resuelve las llaves foráneas con Lookup e inserta solo las observaciones nuevas.
6. **SQL Registrar resumen:** guarda en ResumenCargas los conteos tomados con Row Count.

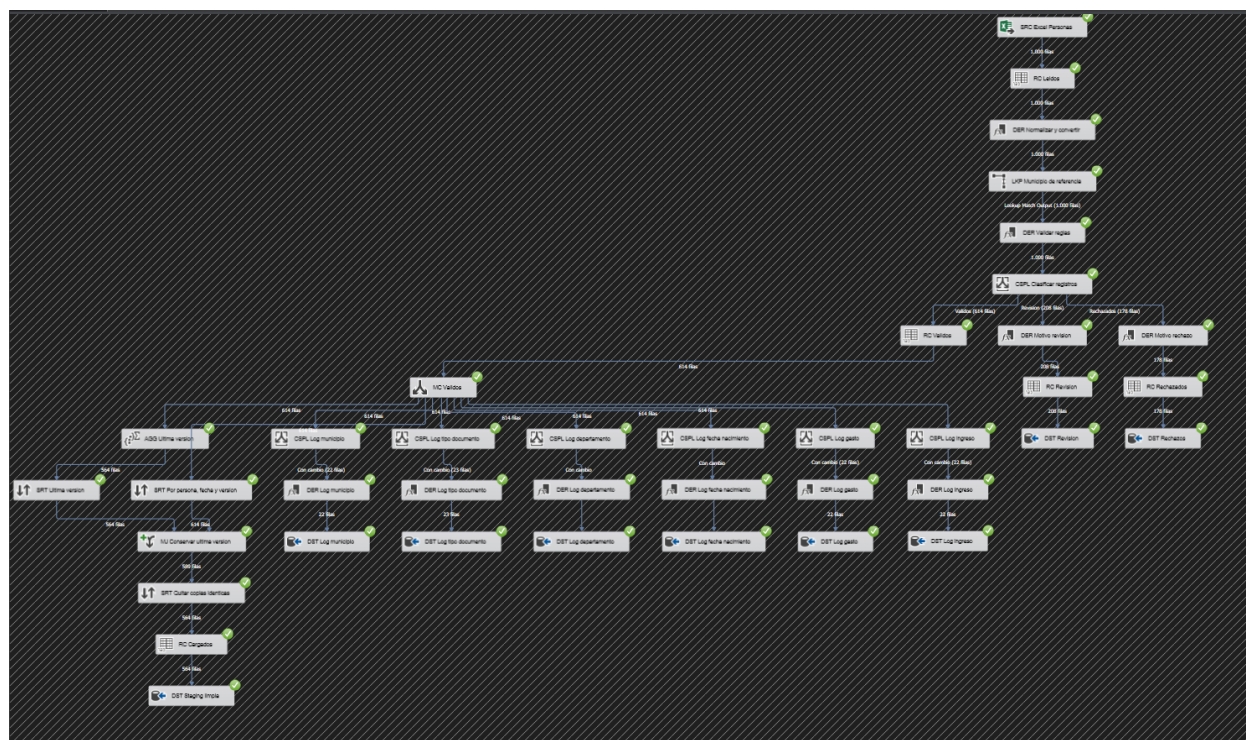
Figura 3*Flujo de control del Paquete A en Visual Studio***Flujo de datos 1: limpieza y validación****Tabla 7***Componentes del flujo de datos DFT 1*

Componente	Tipo	Qué hace
SRC Excel Personas	Excel Source	Lee la hoja Personas del Excel original (todas las columnas como texto)
RC Leídos	Row Count	Cuenta las filas leídas (variable RcLeídos)

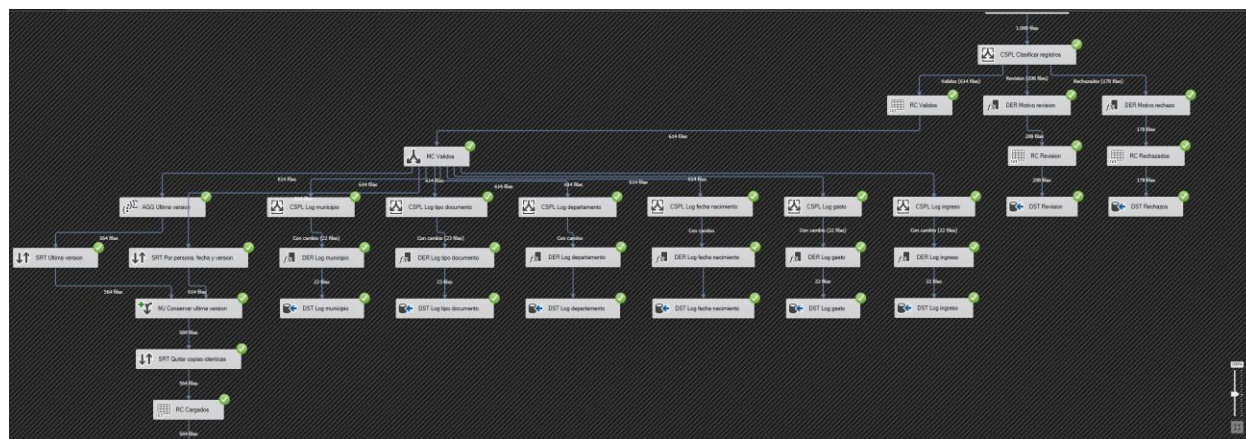
Componente	Tipo	Qué hace
DER Normalizar y convertir	Derived Column	TRIM, mayúsculas, nombres propios, listas de valores válidos, fechas dd/mm/aaaa, importes y conversión de tipos. Si una conversión falla, el valor queda NULL (Ignore failure)
LKP Municipio de referencia	Lookup	Busca el municipio en el catálogo y trae su departamento correcto y su nombre estandarizado
DER Validar reglas	Derived Column	Calcula el balance y arma el motivo de rechazo y el motivo de revisión
CSPL Clasificar registros	Conditional Split	Tres salidas: Rechazados, Revision y Validos
DER Motivo + RC + DST	Derived Column, Row Count, OLE DB Destination	Guardan los rechazos y la revisión en la tabla Rechazos con su motivo
MC Validos	Multicast	Envía los válidos al tratamiento de duplicados y al registro de cambios
CSPL Log ... / DER Log ... / DST Log ...	Conditional Split, Derived Column, OLE DB Destination	Uno por tipo de cambio: filtran las filas corregidas y guardan campo, valor anterior, valor nuevo y regla en LogCambios
SRT Por persona, fecha y versión	Sort	Ordena por tipo y número de documento, fecha de encuesta y fecha de actualización
AGG Última versión	Aggregate	Calcula la fecha de actualización más reciente de cada persona y fecha de encuesta
SRT Última versión	Sort	Ordena el resultado del Aggregate para el Merge Join
MJ Conservar última versión	Merge Join	Unión interna por las cuatro claves: deja solo la versión más reciente
SRT Quitar copias idénticas	Sort (eliminar duplicados)	Elimina las copias exactas que quedan con la misma clave
RC Cargados + DST Staging limpia	Row Count, OLE DB Destination	Cuenta y guarda los registros limpios y únicos en StgPersonas

Figura 4

Flujo de datos DFT 1 ejecutado, con la cantidad de filas en cada camino

**Figura 5**

Acercamiento al DFT 1: clasificación, registro de cambios y tratamiento de duplicados



Nota. Del Conditional Split salen 614 válidos, 208 en revisión y 178 rechazados. El Merge Join deja 589 filas (descarta las 25 versiones viejas) y el Sort elimina las 25 copias idénticas: llegan 564 a la staging.

Flujos de datos 2, 3 y 4: catálogos, personas y observaciones

El **DFT 2** lee la staging, la reparte con un Multicast y usa un Aggregate por catálogo para obtener los valores distintos; un Lookup con salida de "no coincidencia" deja pasar solo los que

todavía no existen, que se insertan con un OLE DB Destination. En ocupaciones, un Conditional Split descarta el grupo sin ocupación.

El **DFT 3** ordena la staging por documento eliminando repetidos (Sort), verifica con un Lookup que la persona no exista e inserta en Personas.

El **DFT 4** resuelve las llaves foráneas con cinco Lookup en cadena (persona, municipio, nivel educativo, situación laboral y ocupación; esta última admite NULL) y un sexto Lookup comprueba que la observación (persona + fecha de encuesta) no exista antes de insertarla. Estos Lookup de "solo los nuevos" son los que impiden que una segunda ejecución genere duplicados.

Figura 6
Flujo de datos DFT 4 (carga de observaciones)



Paquete SSIS B: modelo relacional al data warehouse

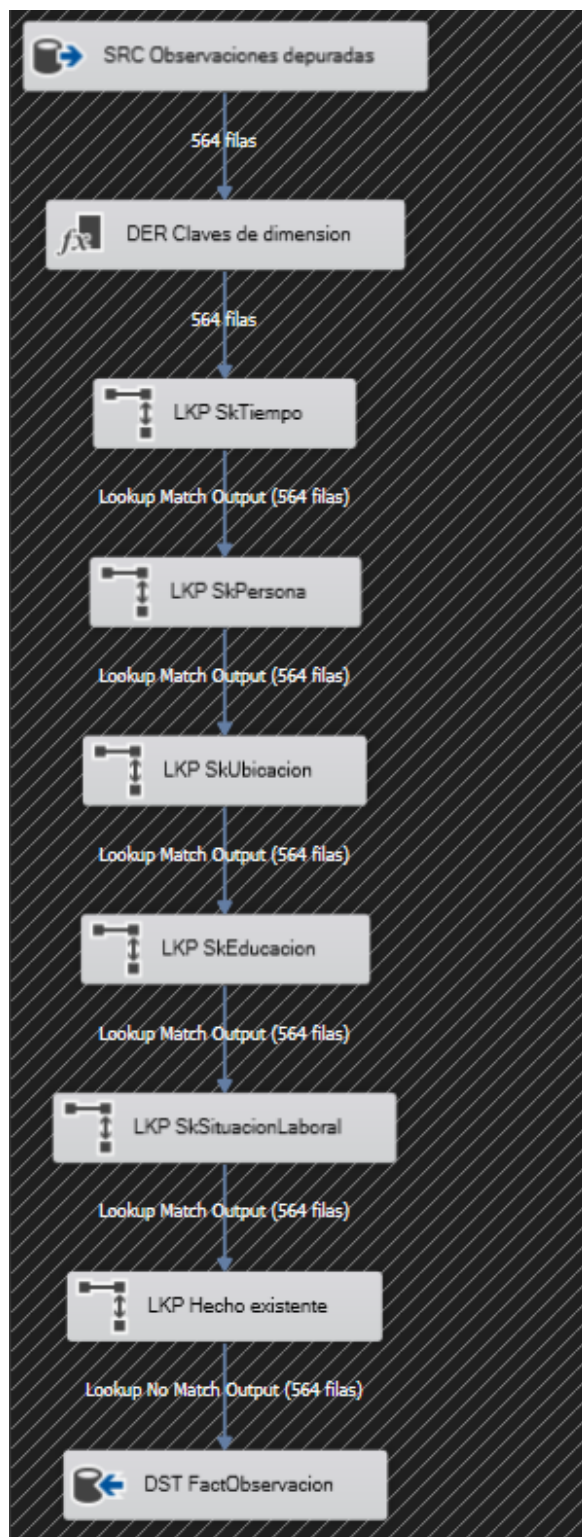
El paquete PaqueteB_RelacionalADW.dtsx toma **exclusivamente la información depurada de `PersonasMER`**. Primero carga las cinco dimensiones (pueden correr en paralelo) y, cuando todas terminan, carga la tabla de hechos.

- **DFT Dim Tiempo:** Aggregate para obtener las fechas de encuesta distintas, Derived Column para año, mes, nombre del mes, trimestre y día, Lookup de fechas existentes y destino.
- **DFT Dim Persona:** Derived Column que arma el nombre completo, Lookup por documento y destino.
- **DFT Dim Ubicación:** Derived Column que pone SIN DATO cuando la zona es NULL, Aggregate de combinaciones distintas de municipio, departamento y zona, Lookup y destino.
- **DFT Dim Educación y DFT Dim Situación Laboral:** Lookup de valores nuevos; en situación laboral, Derived Column con NO APLICA para quien no tiene ocupación y Aggregate de combinaciones distintas.
- **DFT Fact Observación:** Derived Column con los mismos valores por defecto de las dimensiones y la medida CantidadObservaciones = 1; cinco Lookup que reemplazan las claves de negocio por las claves sustitutas; un último Lookup que solo deja pasar hechos nuevos (persona + fecha).

Figura 7

Flujo de control del Paquete B en Visual Studio



Figura 8*Flujo de datos de la tabla de hechos*

En los dos paquetes las transformaciones se hacen con componentes de SSIS (Derived Column, Conditional Split, Lookup, Aggregate, Sort, Merge Join, Multicast y Row Count). No se usó Script Component, ni scripts externos, ni procedimientos almacenados: las únicas instrucciones SQL son las consultas de lectura de los orígenes y las dos tareas de preparación y resumen.

Evidencias de los datos antes y después del tratamiento

Tabla 8

Ejemplos de registros antes (Excel) y después (PersonasMER)

Fila		Tipo doc.	Documento	Nombre	Apellido	Municipio	Ingreso	Gasto	Balance
4	Antes	cc	SIM0000004	Laura	López	ENVIGADO	13900000	3700000	—
	Después	CC	SIM0000004	Laura	López	ENVIGADO	13.900.000,00	3.700.000,00	10.200.000,00
15	Antes	PA	SIM0000015	Camila	Muñoz	cali	7000000	6900000	—
	Después	PA	SIM0000015	Camila	Muñoz	CALI	7.000.000,00	6.900.000,00	100.000,00
18	Antes	CC	SIM0000018	Valentina	Gómez	DUITAMA	\$ 14.700.000,00	2600000	—
	Después	CC	SIM0000018	Valentina	Gómez	DUITAMA	14.700.000,00	2.600.000,00	12.100.000,00
22	Antes	CC	SIM0000022	Valentina	López	CALI	5300000	8900000,50	—
	Después	CC	SIM0000022	Valentina	López	CALI	5.300.000,00	8.900.000,50	-3.600.000,50

Nota. Fila 4: tipo de documento en minúsculas. Fila 15: municipio en minúsculas. Fila 18: ingreso con símbolo \$ y puntos de miles. Fila 22: gasto con coma decimal y balance negativo válido.

Tabla 9

Cambios registrados en LogCambios

Campo			Cambios	
DocumentType			23	
MonthlyExpenses			22	
MonthlyIncome			22	
MunicipalityName			22	
Fila	Campo	Valor anterior	Valor nuevo	Regla
4	DocumentType	cc	CC	Tipo de documento en mayúsculas
22	MonthlyExpenses	8900000,50	8900000.50	La coma decimal pasa a punto
18	MonthlyIncome	\$ 14.700.000,00	14700000.00	Se quita \$ y puntos de miles; la coma decimal pasa a punto
15	MunicipalityName	cali	CALI	Nombre del municipio estandarizado con el catalogo

Nota. En este dataset las filas con departamento incoherente o fecha dd/mm/aaaa también tienen otros errores (por ejemplo, ingreso en letras o municipio M999), así que terminan en rechazo o revisión y esas correcciones no aparecen entre los cargados.

Tabla 10*Ejemplos de registros rechazados y enviados a revisión*

Fila	Motivo
2	RECHAZO: Sin documento;
3	RECHAZO: Sin documento;
13	RECHAZO: Ingreso no numérico (dos millones);
19	RECHAZO: Ingreso negativo (-500000); Empleado sin contrato;
1	REVISIÓN: Edad reportada (31) no coincide con la calculada (19);
5	REVISIÓN: Fecha de nacimiento inválida (31/02/1990);
6	REVISIÓN: Nacimiento posterior a la encuesta;
7	REVISIÓN: Municipio desconocido (M999);

Cantidades de registros aceptados, rechazados y descartados**Tabla 11***Conteos del Paquete A (variables Row Count guardadas en ResumenCargas)*

Leídos	Rechazados	En revisión	Válidos	Duplicados descartados	Cargados
1000	178	208	614	50	564

Nota. 1.000 leídos = 178 rechazados + 208 en revisión + 614 válidos. De los válidos se descartan 50 como duplicados y se cargan 564.

Tabla 12*Motivos de rechazo y de revisión*

Motivo	Tipo	Registros
Sin documento	RECHAZO	68
Sin primer nombre	RECHAZO	22
Ingreso no numérico	RECHAZO	44
Ingreso negativo	RECHAZO	22
Gasto faltante	RECHAZO	22
Fecha de encuesta inválida	RECHAZO	22
Municipio desconocido	REVISIÓN	45
Fecha de nacimiento inválida	REVISIÓN	45
Nacimiento posterior	REVISIÓN	23
no coincide con la calculada	REVISIÓN	46
Empleado sin contrato	REVISIÓN	49

Nota. Un registro puede tener varios motivos, por eso la suma supera a 386.

Tratamiento de duplicados

Las filas 951 a 1000 repiten las filas 801 a 850. En 25 casos la copia es idéntica y en otros 25 cambia el ingreso y la hora de actualización (12:30 frente a 18:45). El ejemplo de la persona SIM0000826 muestra que se conserva la versión más reciente:

Tabla 13

Duplicados en el Excel

Fila	Tipo	Documento	Fecha encuesta	Ingreso	UpdatedAt
826	CC	SIM0000826	2025-07-02	14700000	2025-07-02 12:30:00
976	CC	SIM0000826	2025-07-02	15200000	2025-07-02 18:45:00
801	PA	SIM0000801	2025-03-27	2100000	2025-03-27 12:30:00
951	PA	SIM0000801	2025-03-27	2100000	2025-03-27 12:30:00

Tabla 14

Versión que quedó cargada en PersonasMER

Fila de origen	Documento	Fecha encuesta	Ingreso	Fecha actualización
801	SIM0000801	2025-03-27	2.100.000,00	2025-03-27 12:30:00
976	SIM0000826	2025-07-02	15.200.000,00	2025-07-02 18:45:00

Nota. SIM0000826: se descartó la fila 826 (12:30) y quedó la 976 (18:45, ingreso 15.200.000). SIM0000801: las dos copias eran idénticas y quedó una sola.

Tabla 15

Filas cargadas en cada tabla

Tabla	Filas	Tabla	Filas
Departamentos	5	DimTiempo	321
Municipios	10	DimPersona	564
NivelesEducativos	7	DimUbicacion	29
SituacionesLaborales	5	DimEducacion	7
Ocupaciones	9	DimSituacionLaboral	32
Personas	564	FactObservacion	564
Observaciones	564		
Rechazos	386		
LogCambios	89		

Consultas sobre el data warehouse

Las cuatro consultas están en `sql/04_Consultas.sql`. Cada una une la tabla de hechos con una dimensión y agrupa.

Personas distintas por municipio

```
SELECT u.Departamento, u.Municipio, COUNT(DISTINCT f.SkPersona) AS PersonasDistintas
FROM dbo.FactObservacion f
JOIN dbo.DimUbicacion u ON u.SkUbicacion = f.SkUbicacion
GROUP BY u.Departamento, u.Municipio
ORDER BY u.Departamento, u.Municipio;
```

Tabla 16

Resultado: personas distintas por municipio

Departamento	Municipio	Personas distintas
ANTIOQUIA	ENVIGADO	59
ANTIOQUIA	MEDELLÍN	57
BOGOTÁ D.C.	BOGOTÁ D.C.	61
BOYACÁ	DUITAMA	56
BOYACÁ	TUNJA	55
CUNDINAMARCA	CAJICÁ	49
CUNDINAMARCA	CHÍA	59
CUNDINAMARCA	SOACHA	62
CUNDINAMARCA	ZIPAQUIRÁ	49
VALLE DEL CAUCA	CALI	57

Ingreso promedio por nivel educativo

```
SELECT e.NivelEducativo, CAST(AVG(f.IngresoMensual) AS DECIMAL(18,2)) AS IngresoPromedio,
SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimEducacion e ON e.SkEducacion = f.SkEducacion
GROUP BY e.NivelEducativo
ORDER BY IngresoPromedio DESC;
```

Tabla 17

Resultado: ingreso promedio por nivel educativo

Nivel educativo	Ingreso promedio (COP)	Observaciones
SECUNDARIA	8.369.879,52	83
NINGUNO	8.226.804,12	97
TECNOLOGO	7.702.816,90	71
UNIVERSITARIO	7.685.897,44	78
PRIMARIA	7.641.111,11	90
TECNICO	7.530.769,23	78
POSGRADO	7.208.955,22	67

Balance promedio por situación laboral

```
SELECT s.SituacionLaboral, CAST(AVG(f.BalanceMensual) AS DECIMAL(18,2)) AS BalancePromedio,
SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimSituacionLaboral s ON s.SkSituacionLaboral = f.SkSituacionLaboral
GROUP BY s.SituacionLaboral
ORDER BY BalancePromedio DESC;
```

Tabla 18*Resultado: balance promedio por situación laboral*

Situación laboral	Balance promedio (COP)	Observaciones
EMPLEADO	1.812.096,75	124
DESEMPLEADO	1.784.745,75	118
INDEPENDIENTE	1.708.737,85	103
PENSIONADO	1.152.419,34	124
ESTUDIANTE	1.090.526,29	95

Cantidad de observaciones por año y mes

```

SELECT t.Anio, t.Mes, t.NombreMes, SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimTiempo t ON t.SkTiempo = f.SkTiempo
GROUP BY t.Anio, t.Mes, t.NombreMes
ORDER BY t.Anio, t.Mes;

```

Tabla 19*Resultado: cantidad de observaciones por año y mes*

Año	Mes	Nombre del mes	Observaciones
2025	1	Enero	35
2025	2	Febrero	32
2025	3	Marzo	40
2025	4	Abril	37
2025	5	Mayo	34
2025	6	Junio	34
2025	7	Julio	43
2025	8	Agosto	39
2026	1	Enero	29
2026	2	Febrero	33
2026	3	Marzo	32
2026	4	Abril	27
2026	5	Mayo	49
2026	6	Junio	34
2026	7	Julio	33
2026	8	Agosto	33

Verificación contra el modelo relacional

La segunda parte del script calcula las mismas cuatro respuestas directamente sobre PersonasMER y las compara con las del data warehouse fila por fila. Las cuatro comparaciones devolvieron **Coincide**.

Tabla 20*Verificación de las consultas*

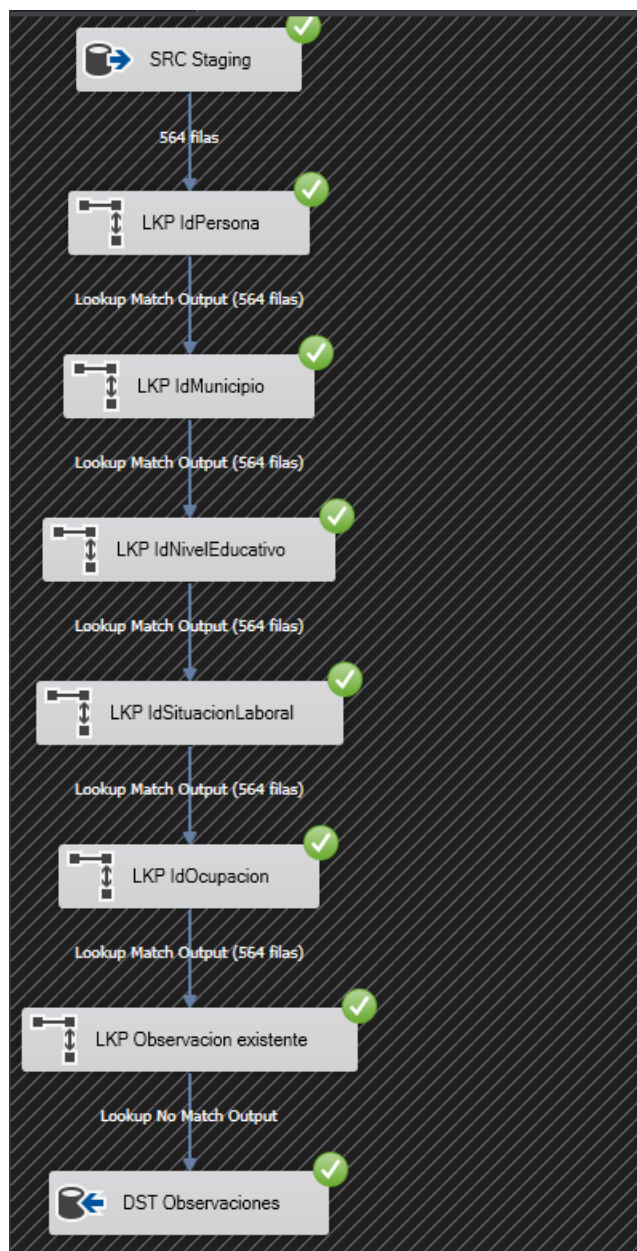
Consulta	Resultado
1. Personas por municipio	Coincide
2. Ingreso promedio por nivel educativo	Coincide
3. Balance promedio por situación laboral	Coincide
4. Observaciones por año y mes	Coincide

Segunda ejecución: los paquetes no generan duplicados

Se ejecutaron nuevamente los dos paquetes sobre las bases ya cargadas. El Paquete A volvió a leer, limpiar y clasificar los 1.000 registros con los mismos resultados, pero los Lookup de "solo los nuevos" no dejaron pasar ninguna persona ni observación repetida; lo mismo ocurrió con las dimensiones y la tabla de hechos del Paquete B. Las restricciones UNIQUE de las tablas son una segunda barrera.

Figura 9

DFT 4 en la segunda ejecución: el Lookup de observaciones existentes no deja pasar ninguna fila



Nota. Las 564 observaciones ya existen, así que la salida "Lookup No Match Output" queda vacía y el destino escribe 0 filas.

Tabla 21

Conteos de cada ejecución del Paquete A

Ejecución	Fecha	Leídos	Rechazados	Revisión	Válidos	Duplicados	Cargados
1	2026-10-02 15:38:20	1000	178	208	614	50	564
2	2026-10-02 15:39:51	1000	178	208	614	50	564

Tabla 22*Filas en las tablas después de la segunda ejecución*

Tabla	Después de la 1. ^a ejecución	Después de la 2. ^a ejecución
PersonasMER.Personas	564	564
PersonasMER.Observaciones	564	564
PersonasDW.DimPersona	564	564
PersonasDW.DimTiempo	321	321
PersonasDW.FactObservacion	564	564

Conclusiones

De los 1.000 registros del Excel, 564 llegaron limpios y sin duplicados al modelo relacional y al data warehouse. Los 386 restantes quedaron identificados con su motivo, sin inventar documentos, fechas ni importes, y los 50 duplicados se resolvieron conservando la versión más reciente.

El cursor sirvió para entender, fila por fila, la lógica de limpieza; sin embargo, SSIS permitió aplicar las mismas reglas a todo el archivo con componentes visuales, contar cada camino y repetir la carga sin duplicar datos. Separar persona y observación en el modelo relacional, y fijar el grano de la tabla de hechos, fue lo que permitió que las consultas del data warehouse coincidieran exactamente con el modelo relacional.

Referencias

Kimball, R., y Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.^a ed.). Wiley.

Microsoft. (2024). *DECLARE CURSOR (Transact-SQL)*. Microsoft Learn.

<https://learn.microsoft.com/es-es/sql/t-sql/language-elements/declare-cursor-transact-sql>

Microsoft. (2024). *Data flow* [SQL Server Integration Services]. Microsoft Learn.

<https://learn.microsoft.com/es-es/sql/integration-services/data-flow/data-flow>

Microsoft. (2024). *Lookup transformation*. Microsoft Learn. [https://learn.microsoft.com/es-](https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/lookup-transformation)

[es/sql/integration-services/data-flow/transformations/lookup-transformation](https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/lookup-transformation)

Microsoft. (2024). *OPENROWSET (Transact-SQL)*. Microsoft Learn.

<https://learn.microsoft.com/es-es/sql/t-sql/functions/openrowset-transact-sql>

Anexo A. Script del modelo relacional

```

/* =====
Actividad 4 - ETL | 01: Modelo relacional (MER)
Base de datos destino del Paquete SSIS A (Excel -> relacional).
Tablas segun el diagrama: diagramas/Diagrama_Modelo_Relacional
===== */

IF DB_ID('PersonasMER') IS NULL
    CREATE DATABASE PersonasMER;
GO

USE PersonasMER;
GO

-- Se borran en orden inverso a las llaves foraneas para poder re-ejecutar el script
DROP TABLE IF EXISTS dbo.StgPersonas;
DROP TABLE IF EXISTS dbo.ResumenCargas;
DROP TABLE IF EXISTS dbo.Observaciones;
DROP TABLE IF EXISTS dbo.Personas;
DROP TABLE IF EXISTS dbo.Municipios;
DROP TABLE IF EXISTS dbo.Departamentos;
DROP TABLE IF EXISTS dbo.NivelesEducativos;
DROP TABLE IF EXISTS dbo.SituacionesLaborales;
DROP TABLE IF EXISTS dbo.Ocupaciones;
DROP TABLE IF EXISTS dbo.Rechazos;
DROP TABLE IF EXISTS dbo.LogCambios;
GO

/* ----- Catalogos ----- */

CREATE TABLE dbo.Departamentos (
    IdDepartamento INT IDENTITY(1,1) CONSTRAINT PK_Departamentos PRIMARY KEY,
    CodigoDepartamento NVARCHAR(10) NOT NULL CONSTRAINT UQ_Departamentos_Codigo UNIQUE,
    NombreDepartamento NVARCHAR(60) NOT NULL
);

CREATE TABLE dbo.Municipios (
    IdMunicipio INT IDENTITY(1,1) CONSTRAINT PK_Municipios PRIMARY KEY,
    IdDepartamento INT NOT NULL
    CONSTRAINT FK_Municipios_Departamentos REFERENCES dbo.Departamentos (IdDepartamento),
    NombreMunicipio NVARCHAR(60) NOT NULL,
    CodigoMunicipio NVARCHAR(10) NOT NULL CONSTRAINT UQ_Municipios_Codigo UNIQUE
);

CREATE TABLE dbo.NivelesEducativos (
    IdNivelEducativo INT IDENTITY(1,1) CONSTRAINT PK_NivelesEducativos PRIMARY KEY,
    NombreNivel NVARCHAR(30) NOT NULL CONSTRAINT UQ_NivelesEducativos_Nombre UNIQUE
);

CREATE TABLE dbo.SituacionesLaborales (
    IdSituacionLaboral INT IDENTITY(1,1) CONSTRAINT PK_SituacionesLaborales PRIMARY KEY,
    NombreSituacion NVARCHAR(30) NOT NULL CONSTRAINT UQ_SituacionesLaborales_Nombre UNIQUE
);

CREATE TABLE dbo.Ocupaciones (
    IdOcupacion INT IDENTITY(1,1) CONSTRAINT PK_Ocupaciones PRIMARY KEY,
    CodigoOcupacion NVARCHAR(10) NOT NULL CONSTRAINT UQ_Ocupaciones_Codigo UNIQUE,
    NombreOcupacion NVARCHAR(60) NOT NULL
);

/* ----- Personas: una por tipo + numero de documento ----- */

CREATE TABLE dbo.Personas (
    IdPersona INT IDENTITY(1,1) CONSTRAINT PK_Personas PRIMARY KEY,
    TipoDocumento NVARCHAR(5) NOT NULL,
    NumeroDocumento NVARCHAR(20) NOT NULL,
    PrimerNombre NVARCHAR(50) NOT NULL,
    SegundoNombre NVARCHAR(50) NULL,

```

```

PrimerApellido      NVARCHAR(50) NOT NULL,
SegundoApellido     NVARCHAR(50) NULL,
FechaNacimiento     DATE NULL,
Sexo                NVARCHAR(2) NULL,
EstadoCivil         NVARCHAR(20) NULL,
Correo              NVARCHAR(100) NULL,
Telefono            NVARCHAR(20) NULL,
CONSTRAINT UQ_Personas_Documento UNIQUE (TipoDocumento, NumeroDocumento)
);

/* ----- Observaciones: una por persona + fecha de encuesta ----- */

CREATE TABLE dbo.Observaciones (
    IdObservacion     INT IDENTITY(1,1) CONSTRAINT PK_Observaciones PRIMARY KEY,
    IdPersona         INT NOT NULL
        CONSTRAINT FK_Observaciones_Personas REFERENCES dbo.Personas (IdPersona),
    IdMunicipio       INT NULL
        CONSTRAINT FK_Observaciones_Municipios REFERENCES dbo.Municipios (IdMunicipio),
    FechaEncuesta     DATE NOT NULL,
    EdadReportada     TINYINT NULL,
    Zona             NVARCHAR(10) NULL,
    Direccion         NVARCHAR(100) NULL,
    TipoVivienda      NVARCHAR(20) NULL,
    Estrato           TINYINT NULL CONSTRAINT CK_Observaciones_Estrato CHECK (Estrato BETWEEN 1
AND 6),
    IdSituacionLaboral INT NOT NULL
        CONSTRAINT FK_Observaciones_SituacionesLaborales REFERENCES dbo.SituacionesLaborales
(IdSituacionLaboral),
    IdNivelEducativo  INT NOT NULL
        CONSTRAINT FK_Observaciones_NivelesEducativos REFERENCES dbo.NivelesEducativos (IdNivelEducativo),
    IdOcupacion       INT NULL
        CONSTRAINT FK_Observaciones_Ocupaciones REFERENCES dbo.Ocupaciones (IdOcupacion),
    Empleador         NVARCHAR(60) NULL,
    TipoContrato      NVARCHAR(20) NULL,
    FechaInicioEmpleo DATE NULL,
    IngresoMensual    DECIMAL(18,2) NOT NULL CONSTRAINT CK_Observaciones_Ingreso CHECK (IngresoMensual
>= 0),
    GastoMensual      DECIMAL(18,2) NOT NULL CONSTRAINT CK_Observaciones_Gasto CHECK (GastoMensual >=
0),
    BalanceMensual    DECIMAL(18,2) NOT NULL, -- puede ser negativo
    PersonasACargo    TINYINT NULL,
    TamanoHogar       TINYINT NULL CONSTRAINT CK_Observaciones_Hogar CHECK (TamanoHogar >= 1),
    RegimenSalud      NVARCHAR(20) NULL,
    Discapacidad      NVARCHAR(2) NULL,
    CanalOrigen       NVARCHAR(20) NULL,
    FechaActualizacion DATETIME NULL,
    SourceRowId       INT NOT NULL,
    CONSTRAINT UQ_Observaciones_PersonaFecha UNIQUE (IdPersona, FechaEncuesta)
);

/* ----- Tablas de control ----- */

CREATE TABLE dbo.Rechazos (
    IdRechazo         INT IDENTITY(1,1) CONSTRAINT PK_Rechazos PRIMARY KEY,
    SourceRowId       INT NOT NULL,
    Motivo            NVARCHAR(400) NOT NULL,
    FechaProceso      DATETIME2(0) NOT NULL CONSTRAINT DF_Rechazos_Fecha DEFAULT SYSDATETIME()
);

CREATE TABLE dbo.LogCambios (
    IdCambio          INT IDENTITY(1,1) CONSTRAINT PK_LogCambios PRIMARY KEY,
    SourceRowId       INT NOT NULL,
    Campo             NVARCHAR(50) NOT NULL,
    ValorAnterior     NVARCHAR(200) NULL,
    ValorNuevo       NVARCHAR(200) NULL,
    Regla             NVARCHAR(200) NOT NULL,
    FechaProceso      DATETIME2(0) NOT NULL CONSTRAINT DF_LogCambios_Fecha DEFAULT SYSDATETIME()
);

-- Conteos de cada ejecucion del Paquete A (los llena el paquete con Row Count)
CREATE TABLE dbo.ResumenCargas (
    IdResumen         INT IDENTITY(1,1) CONSTRAINT PK_ResumenCargas PRIMARY KEY,

```

```

    FechaProceso      DATETIME2(0)    NOT NULL CONSTRAINT DF_ResumenCargas_Fecha DEFAULT SYSDATETIME(),
    Leidos            INT              NOT NULL,
    Rechazados        INT              NOT NULL,
    EnRevision        INT              NOT NULL,
    Validos           INT              NOT NULL,
    DuplicadosDescartados INT          NOT NULL,
    Cargados          INT              NOT NULL
);

/* ----- Staging: datos ya limpios y sin duplicados -----
El Paquete A la llena en el primer flujo y de aqui salen
los catalogos, las personas y las observaciones. */

CREATE TABLE dbo.StgPersonas (
    SourceRowId      INT              NOT NULL,
    TipoDocumento    NVARCHAR(5)     NOT NULL,
    NumeroDocumento  NVARCHAR(20)    NOT NULL,
    PrimerNombre     NVARCHAR(50)    NOT NULL,
    SegundoNombre    NVARCHAR(50)    NULL,
    PrimerApellido   NVARCHAR(50)    NOT NULL,
    SegundoApellido  NVARCHAR(50)    NULL,
    FechaNacimiento  DATE             NULL,
    EdadReportada    INT              NULL,
    Sexo             NVARCHAR(2)      NULL,
    EstadoCivil      NVARCHAR(20)     NULL,
    Correo           NVARCHAR(100)    NULL,
    Telefono         NVARCHAR(20)     NULL,
    CodigoMunicipio  NVARCHAR(10)     NOT NULL,
    Zona            NVARCHAR(10)     NULL,
    Direccion        NVARCHAR(100)    NULL,
    TipoVivienda     NVARCHAR(20)     NULL,
    Estrato          TINYINT          NULL,
    NivelEducativo   NVARCHAR(30)     NOT NULL,
    SituacionLaboral NVARCHAR(30)     NOT NULL,
    CodigoOcupacion  NVARCHAR(10)     NULL,
    NombreOcupacion  NVARCHAR(60)     NULL,
    Empleador        NVARCHAR(60)     NULL,
    TipoContrato     NVARCHAR(20)     NULL,
    FechaInicioEmpleo DATE           NULL,
    IngresoMensual   DECIMAL(18,2)    NOT NULL,
    GastoMensual     DECIMAL(18,2)    NOT NULL,
    BalanceMensual   DECIMAL(18,2)    NOT NULL,
    PersonasACargo   TINYINT          NULL,
    TamanoHogar      TINYINT          NULL,
    RegimenSalud     NVARCHAR(20)     NULL,
    Discapacidad     NVARCHAR(2)      NULL,
    CanalOrigen      NVARCHAR(20)     NULL,
    FechaEncuesta    DATE             NOT NULL,
    FechaActualizacion DATETIME       NULL
);
GO

/* ----- Datos maestros de referencia (catalogo geografico interno) -----
Codigos internos del dataset (no son DIVIPOLA). Se toman de la combinacion
codigo-nombre que trae el Excel. Sirven para validar la coherencia entre
municipio y departamento y para estandarizar el nombre del municipio.
M999 "MUNICIPIO DESCONOCIDO" no se incluye: esas filas van a revision. */

INSERT INTO dbo.Departamentos (CodigoDepartamento, NombreDepartamento) VALUES
    (N'D01', N'CUNDINAMARCA'),
    (N'D02', N'BOGOTÁ D.C.'),
    (N'D03', N'ANTIOQUIA'),
    (N'D04', N'BOYACÁ'),
    (N'D05', N'VALLE DEL CAUCA');

INSERT INTO dbo.Municipios (IdDepartamento, NombreMunicipio, CodigoMunicipio)
SELECT d.IdDepartamento, v.Nombre, v.Codigo
FROM (VALUES
    (N'M001', N'CHÍA', N'D01'),
    (N'M002', N'CAJICÁ', N'D01'),
    (N'M003', N'ZIPAQUIRÁ', N'D01'),
    (N'M004', N'SOACHA', N'D01'),
    (N'M005', N'BOGOTÁ D.C.', N'D02'),

```

```
(N'M006', N'MEDELLÍN', N'D03'),  
(N'M007', N'ENVIGADO', N'D03'),  
(N'M008', N'TUNJA', N'D04'),  
(N'M009', N'DUITAMA', N'D04'),  
(N'M010', N'CALI', N'D05')  
) AS v (Codigo, Nombre, CodDepto)  
JOIN dbo.Departamentos d ON d.CodigoDepartamento = v.CodDepto;  
GO  
  
PRINT 'Modelo relacional PersonasMER creado.';
```

Anexo B. Script del modelo estrella

```

/* =====
Actividad 4 - ETL | 02: Data warehouse en esquema estrella
Destino del Paquete SSIS B (relacional -> DW).
Grano de FactObservacion: una observacion por persona y fecha de encuesta.
Tablas segun el diagrama: diagramas/Diagrama_Modelo_Estrella
===== */

IF DB_ID('PersonasDW') IS NULL
    CREATE DATABASE PersonasDW;
GO

USE PersonasDW;
GO

DROP TABLE IF EXISTS dbo.FactObservacion;
DROP TABLE IF EXISTS dbo.DimTiempo;
DROP TABLE IF EXISTS dbo.DimPersona;
DROP TABLE IF EXISTS dbo.DimUbicacion;
DROP TABLE IF EXISTS dbo.DimEducacion;
DROP TABLE IF EXISTS dbo.DimSituacionLaboral;
GO

/* ----- Dimensiones (claves sustitutas Sk) ----- */

CREATE TABLE dbo.DimTiempo (
    SkTiempo          INT IDENTITY(1,1) CONSTRAINT PK_DimTiempo PRIMARY KEY,
    Fecha             DATE              NOT NULL CONSTRAINT UQ_DimTiempo_Fecha UNIQUE,
    Anio              SMALLINT          NOT NULL,
    Mes               TINYINT           NOT NULL,
    NombreMes         NVARCHAR(15)     NOT NULL,
    Trimestre         TINYINT           NOT NULL,
    Dia               TINYINT           NOT NULL
);

CREATE TABLE dbo.DimPersona (
    SkPersona          INT IDENTITY(1,1) CONSTRAINT PK_DimPersona PRIMARY KEY,
    TipoDocumento      NVARCHAR(5)     NOT NULL,
    NumeroDocumento    NVARCHAR(20)    NOT NULL,
    NombreCompleto     NVARCHAR(210)   NOT NULL,
    Sexo               NVARCHAR(2)     NULL,
    EstadoCivil        NVARCHAR(20)    NULL,
    CONSTRAINT UQ_DimPersona_Documento UNIQUE (TipoDocumento, NumeroDocumento)
);

CREATE TABLE dbo.DimUbicacion (
    SkUbicacion        INT IDENTITY(1,1) CONSTRAINT PK_DimUbicacion PRIMARY KEY,
    CodigoMunicipio    NVARCHAR(10)    NOT NULL,
    Municipio          NVARCHAR(60)    NOT NULL,
    CodigoDepartamento NVARCHAR(10)    NOT NULL,
    Departamento       NVARCHAR(60)    NOT NULL,
    Zona              NVARCHAR(10)    NOT NULL, -- 'SIN DATO' cuando no se conoce
    CONSTRAINT UQ_DimUbicacion UNIQUE (CodigoMunicipio, Zona)
);

CREATE TABLE dbo.DimEducacion (
    SkEducacion        INT IDENTITY(1,1) CONSTRAINT PK_DimEducacion PRIMARY KEY,
    NivelEducativo     NVARCHAR(30)    NOT NULL CONSTRAINT UQ_DimEducacion UNIQUE
);

CREATE TABLE dbo.DimSituacionLaboral (
    SkSituacionLaboral INT IDENTITY(1,1) CONSTRAINT PK_DimSituacionLaboral PRIMARY KEY,
    SituacionLaboral   NVARCHAR(30)    NOT NULL,
    Ocupacion          NVARCHAR(60)    NOT NULL, -- 'NO APLICA' cuando no tiene
    TipoContrato       NVARCHAR(20)    NOT NULL,
    CONSTRAINT UQ_DimSituacionLaboral UNIQUE (SituacionLaboral, Ocupacion, TipoContrato)
);

```

```

/* ----- Tabla de hechos ----- */

CREATE TABLE dbo.FactObservacion (
    UniqueID          INT IDENTITY(1,1) CONSTRAINT PK_FactObservacion PRIMARY KEY,
    SkTiempo           INT NOT NULL CONSTRAINT FK_Fact_DimTiempo      REFERENCES dbo.DimTiempo
(SkTiempo),
    SkPersona          INT NOT NULL CONSTRAINT FK_Fact_DimPersona    REFERENCES dbo.DimPersona
(SkPersona),
    SkUbicacion        INT NOT NULL CONSTRAINT FK_Fact_DimUbicacion  REFERENCES dbo.DimUbicacion
(SkUbicacion),
    SkEducacion        INT NOT NULL CONSTRAINT FK_Fact_DimEducacion  REFERENCES dbo.DimEducacion
(SkEducacion),
    SkSituacionLaboral INT NOT NULL CONSTRAINT FK_Fact_DimSituacionLaboral REFERENCES
dbo.DimSituacionLaboral (SkSituacionLaboral),
    IngresoMensual     DECIMAL(18,2) NOT NULL,
    GastoMensual       DECIMAL(18,2) NOT NULL,
    BalanceMensual     DECIMAL(18,2) NOT NULL,
    CantidadObservaciones INT NOT NULL CONSTRAINT DF_Fact_Cantidad DEFAULT 1,
    -- El grano: una fila por persona y fecha de encuesta (evita duplicados al re-ejecutar)
    CONSTRAINT UQ_FactObservacion_Grano UNIQUE (SkPersona, SkTiempo)
);
GO

PRINT 'Data warehouse PersonasDW creado.';

```

Anexo C. Script de consultas y verificación

```

/* =====
Actividad 4 - ETL | 04: Consultas sobre el data warehouse
Cada consulta del DW se compara con la misma pregunta hecha sobre el
modelo relacional: los resultados deben coincidir.
===== */

USE PersonasDW;
GO

/* ----- 1. Personas distintas por municipio ----- */
SELECT u.Departamento, u.Municipio, COUNT(DISTINCT f.SkPersona) AS PersonasDistintas
FROM dbo.FactObservacion f
JOIN dbo.DimUbicacion u ON u.SkUbicacion = f.SkUbicacion
GROUP BY u.Departamento, u.Municipio
ORDER BY u.Departamento, u.Municipio;

/* ----- 2. Ingreso promedio por nivel educativo ----- */
SELECT e.NivelEducativo, CAST(AVG(f.IngresoMensual) AS DECIMAL(18,2)) AS IngresoPromedio,
SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimEducacion e ON e.SkEducacion = f.SkEducacion
GROUP BY e.NivelEducativo
ORDER BY IngresoPromedio DESC;

/* ----- 3. Balance promedio por situacion laboral ----- */
SELECT s.SituacionLaboral, CAST(AVG(f.BalanceMensual) AS DECIMAL(18,2)) AS BalancePromedio,
SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimSituacionLaboral s ON s.SkSituacionLaboral = f.SkSituacionLaboral
GROUP BY s.SituacionLaboral
ORDER BY BalancePromedio DESC;

/* ----- 4. Cantidad de observaciones por año y mes ----- */
SELECT t.Año, t.Mes, t.NombreMes, SUM(f.CantidadObservaciones) AS Observaciones
FROM dbo.FactObservacion f
JOIN dbo.DimTiempo t ON t.SkTiempo = f.SkTiempo
GROUP BY t.Año, t.Mes, t.NombreMes
ORDER BY t.Año, t.Mes;
GO

/* =====
Verificacion: la misma informacion calculada desde el modelo relacional.
Si el DW esta bien cargado, cada comparacion devuelve "Coincide".
===== */

-- 1. Personas por municipio
WITH dw AS (
    SELECT u.CodigoMunicipio, COUNT(DISTINCT f.SkPersona) AS n
    FROM PersonasDW.dbo.FactObservacion f JOIN PersonasDW.dbo.DimUbicacion u ON u.SkUbicacion =
f.SkUbicacion
    GROUP BY u.CodigoMunicipio),
mer AS (
    SELECT m.CodigoMunicipio, COUNT(DISTINCT o.IdPersona) AS n
    FROM PersonasMER.dbo.Observaciones o JOIN PersonasMER.dbo.Municipios m ON m.IdMunicipio =
o.IdMunicipio
    GROUP BY m.CodigoMunicipio)
SELECT '1. Personas por municipio' AS Consulta,
    CASE WHEN EXISTS (SELECT * FROM dw FULL JOIN mer ON mer.CodigoMunicipio = dw.CodigoMunicipio
        WHERE dw.n IS NULL OR mer.n IS NULL OR dw.n <> mer.n)
    THEN 'NO coincide' ELSE 'Coincide' END AS Resultado
UNION ALL
-- 2. Ingreso promedio por nivel educativo
SELECT '2. Ingreso promedio por nivel educativo',
    CASE WHEN EXISTS (
        SELECT * FROM
        (SELECT e.NivelEducativo AS k, AVG(f.IngresoMensual) AS v FROM
PersonasDW.dbo.FactObservacion f

```

```

        JOIN PersonasDW.dbo.DimEducacion e ON e.SkEducacion = f.SkEducacion GROUP BY
e.NivelEducativo) dw
        FULL JOIN
        (SELECT n.NombreNivel AS k, AVG(o.IngresoMensual) AS v FROM PersonasMER.dbo.Observaciones o
        JOIN PersonasMER.dbo.NivelesEducativos n ON n.IdNivelEducativo = o.IdNivelEducativo GROUP
BY n.NombreNivel) mer
        ON mer.k = dw.k WHERE dw.v IS NULL OR mer.v IS NULL OR dw.v <> mer.v)
        THEN 'NO coincide' ELSE 'Coincide' END
UNION ALL
-- 3. Balance promedio por situacion laboral
SELECT '3. Balance promedio por situacion laboral',
        CASE WHEN EXISTS (
                SELECT * FROM
                (SELECT s.SituacionLaboral AS k, AVG(f.BalanceMensual) AS v FROM
PersonasDW.dbo.FactObservacion f
                JOIN PersonasDW.dbo.DimSituacionLaboral s ON s.SkSituacionLaboral = f.SkSituacionLaboral
GROUP BY s.SituacionLaboral) dw
                FULL JOIN
                (SELECT s.NombreSituacion AS k, AVG(o.BalanceMensual) AS v FROM
PersonasMER.dbo.Observaciones o
                JOIN PersonasMER.dbo.SituacionesLaborales s ON s.IdSituacionLaboral = o.IdSituacionLaboral
GROUP BY s.NombreSituacion) mer
                ON mer.k = dw.k WHERE dw.v IS NULL OR mer.v IS NULL OR dw.v <> mer.v)
                THEN 'NO coincide' ELSE 'Coincide' END
UNION ALL
-- 4. Observaciones por año y mes
SELECT '4. Observaciones por año y mes',
        CASE WHEN EXISTS (
                SELECT * FROM
                (SELECT t.Anio * 100 + t.Mes AS k, SUM(f.CantidadObservaciones) AS v FROM
PersonasDW.dbo.FactObservacion f
                JOIN PersonasDW.dbo.DimTiempo t ON t.SkTiempo = f.SkTiempo GROUP BY t.Anio * 100 + t.Mes)
dw
                FULL JOIN
                (SELECT YEAR(o.FechaEncuesta) * 100 + MONTH(o.FechaEncuesta) AS k, COUNT(*) AS v FROM
PersonasMER.dbo.Observaciones o
                GROUP BY YEAR(o.FechaEncuesta) * 100 + MONTH(o.FechaEncuesta)) mer
                ON mer.k = dw.k WHERE dw.v IS NULL OR mer.v IS NULL OR dw.v <> mer.v)
                THEN 'NO coincide' ELSE 'Coincide' END;
GO

/* =====
Prueba de no duplicados: correr esto, ejecutar otra vez los dos
paquetes y volver a correrlo. Los conteos deben quedar iguales.
===== */
SELECT 'MER Personas' AS Tabla, COUNT(*) AS Filas FROM PersonasMER.dbo.Personas
UNION ALL SELECT 'MER Observaciones', COUNT(*) FROM PersonasMER.dbo.Observaciones
UNION ALL SELECT 'DW DimTiempo', COUNT(*) FROM PersonasDW.dbo.DimTiempo
UNION ALL SELECT 'DW DimPersona', COUNT(*) FROM PersonasDW.dbo.DimPersona
UNION ALL SELECT 'DW DimUbicacion', COUNT(*) FROM PersonasDW.dbo.DimUbicacion
UNION ALL SELECT 'DW DimEducacion', COUNT(*) FROM PersonasDW.dbo.DimEducacion
UNION ALL SELECT 'DW DimSituacionLaboral', COUNT(*) FROM PersonasDW.dbo.DimSituacionLaboral
UNION ALL SELECT 'DW FactObservacion', COUNT(*) FROM PersonasDW.dbo.FactObservacion;

-- Conteos de cada ejecucion del Paquete A
SELECT * FROM PersonasMER.dbo.ResumenCargas ORDER BY IdResumen;

```